



JAVA TECHNOLOGIES

SUBJECT CODE : UQ24CA251B

Samyukta D Kumta
Computer Applications

Course Content:

Unit 4: GUI Programming using Swing

Introduction to GUI Programming, Components of GUI; Swing: Introduction, Features of Swing, Swing Containers, Swing Components, Mouse and Keyboard Events, A Simple Swing Application, Swing Application with JDBC

Experiential Learning: Developing GUI Application using Swing, Developing Database Application using Swing and MySQL

Introduction to GUI Programming in Java

GUI (Graphical User Interface) programming is used to create applications that interact with users through graphical components instead of text commands.

Examples of GUI elements:

- Buttons
- Text fields
- Windows
- Menus
- Dialog boxes

JAVA TECHNOLOGIES

Introduction to GUI Programming in Java

In Java, GUI applications are mainly developed using:

- AWT (Abstract Window Toolkit)
- Swing

Swing is more advanced and widely used.

Example

A login window with:

- Username field
- Password field
- Login button

Instead of typing commands in the console, users interact using graphical components.

Introduction to GUI Programming in Java

Components of GUI

A GUI consists of several visual elements called components.

Main GUI Components

1. Window / Frame

- Main container that holds other components.
- Implemented using JFrame.

Syntax:

```
JFrame frame = new JFrame("My Application");
```

Introduction to GUI Programming in Java

2. Label

Displays text or images.

Syntax:

```
JLabel label = new JLabel("Enter Name:");
```

3. Text Field

Used to input text.

Syntax:

```
JTextField tf = new JTextField(20);
```

Introduction to GUI Programming in Java

4. Button

Triggers an action when clicked.

Syntax:

```
JButton button = new JButton("Submit");
```

5. Checkbox

Allows selection of multiple options.

Syntax:

```
JCheckBox cb = new JCheckBox("Accept Terms");
```

Introduction to GUI Programming in Java

6. Radio Button

Allows selection of only one option.

Syntax:

```
JRadioButton r1 = new JRadioButton("Male");
```

7. Menu

Provides options in the application menu bar.

Syntax:

```
JMenu menu = new JMenu("File");
```

AWT

- AWT (Abstract Window Toolkit) is Java's original set of classes used to create Graphical User Interfaces (GUI).
- AWT provides basic GUI components like buttons, labels, text fields, and windows.

Key Features of AWT

- **Platform Dependent** – Uses OS-specific components
- **Heavyweight Components** – Depends on native system resources
- **Basic GUI Controls** – Limited number of components
- **Event Handling Support** – Uses listeners for interaction
- **Simple and Fast** – Less complex compared to Swing

AWT

Advantages of AWT

- Simple to use
- Faster for small applications
- Direct interaction with OS

Disadvantages of AWT

- Platform dependent
- Limited components
- Less flexible UI
- Not suitable for modern applications

Swing

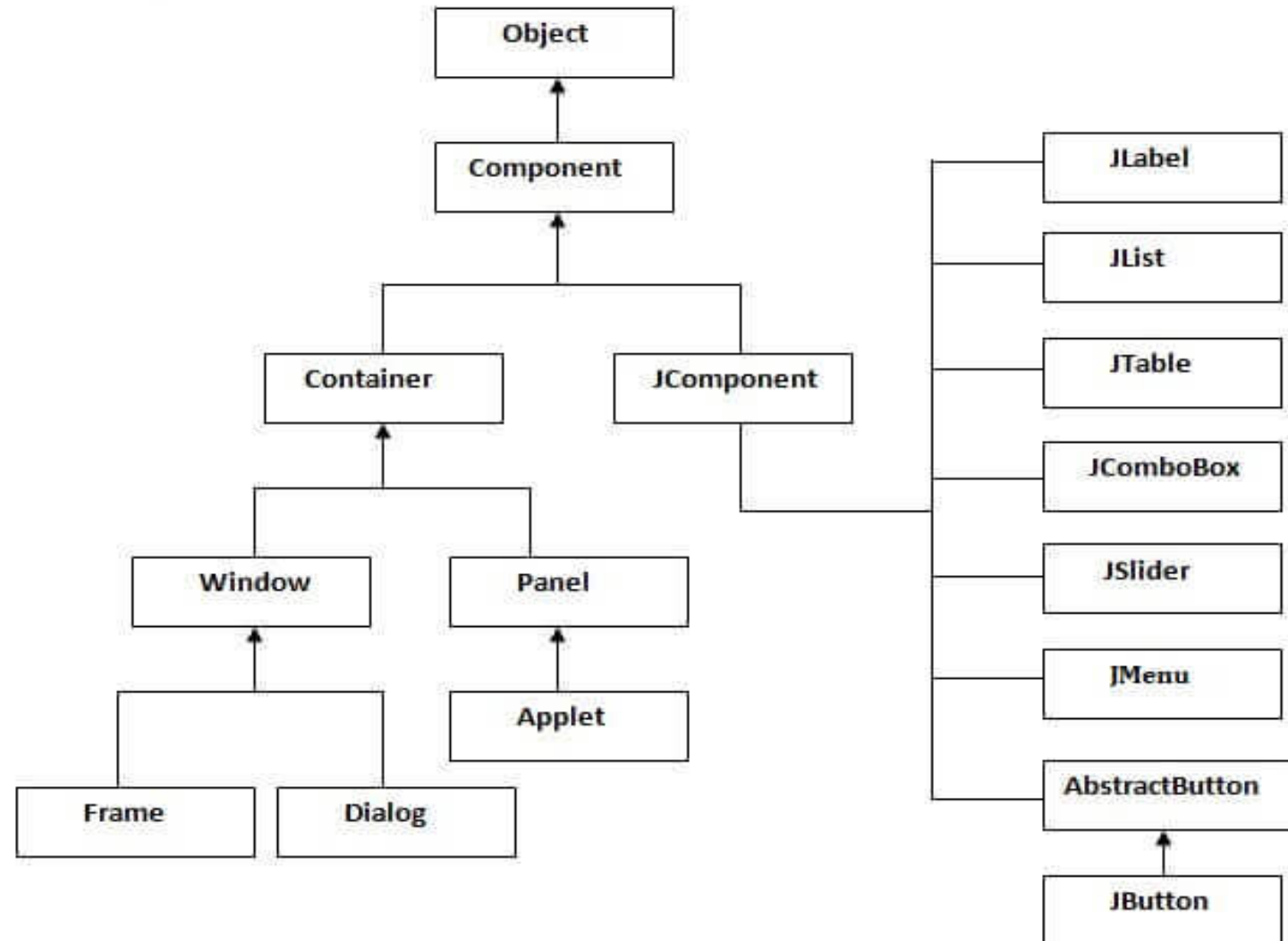
- Swing in Java is a framework, i.e. a pre-coded toolkit used by Java developers to create Java applications or web applications.
- All the swing classes are present in javax.swing package.
- Java Swing Framework is part of Java Foundation Classes (JFC); JFC is a set of libraries that supports programmers in creating applications using Java.
- Swing in Java is built on top of AWT (Abstract Windowing Toolkit) Application Program Interface (API) and is entirely written in Java

Swing

What is JFC?

The Java Foundation Classes (JFC) represent a comprehensive toolkit for building desktop applications with Java. While many perceive JFC and Swing as synonymous, JFC encompasses a broader range of functionalities beyond Swing.

Hierarchy of JAVA Swing Class



Commonly Used Methods of Component Class

The methods of a component class are widely used in Java Swing, which are given below.

- **public void add(Component c):** It adds a component on another component.
- **public void setSize(int width,int height):** It sets size of the component.
- **public void setLayout(LayoutManager m):** It sets the layout manager for the component.
- **public void setVisible(boolean b):** It sets the visibility of the component. It is, by default, false.

JAVA TECHNOLOGIES

Swing Vs AWT

Feature	AWT	Swing
Type	Heavyweight	Lightweight
Platform	Dependent	Independent
Components	Limited	Rich set
Look & Feel	OS-based	Customizable

Key Features of Swing

- **Platform Independent** – Works the same on all operating systems
- **Lightweight Components** – Not dependent on OS components
- **Rich Set of Controls** – JButton, JTable, JTextArea, etc.
- **Pluggable Look and Feel** – Can change appearance at runtime
- **MVC Architecture** – Separates data, UI, and control logic
- **Highly Customizable** – Fonts, colors, layouts

JAVA TECHNOLOGIES

Basic Swing Components

Swing components are the building blocks of a GUI application in Java. They are used to create user interfaces such as buttons, labels, text fields, tables, etc.

All Swing components are part of the package:

`javax.swing`

Basic Swing Components

1. Containers

Containers hold and organize other components.

Top-Level Containers

- JFrame – Main window
- JDialog – Dialog box
- JApplet – Applet (deprecated)

Intermediate Containers

- JPanel – Group components
- JScrollPane – Adds scrolling
- JTabbedPane – Tabs interface

2. Basic Components/Controls

Component	Description
JLabel	Displays text or images
JButton	Performs an action when clicked
TextField	Single-line text input
TextArea	Multi-line text input
JCheckBox	Allows multiple selections
JRadioButton	Allows single selection
JComboBox	Dropdown list
JList	Displays list of items

3. Advanced Components

Component	Description
JTable	Displays data in tabular form
JTree	Displays hierarchical data
JMenuBar, JMenu, JMenuItem	Menu system
JProgressBar	Shows progress
JSlider	Select value by sliding

Key Characteristics of Swing Components

- Begin with letter “J” (e.g., JButton, JLabel)
- Lightweight (no OS dependency)
- Customizable (fonts, colors, sizes)
- Follow MVC architecture
- Support event handling

1. JFrame (Main Window)

Syntax :

```
JFrame frame = new JFrame("Title");
```

```
frame.setSize(width, height);
```

```
frame.setLayout(null);
```

```
frame.setVisible(true);
```

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

2. Jframe : Methods

Method	Purpose
setSize(int width, int height)	Sets the size of the frame
setTitle(String title)	Sets the title of the window
setLayout(LayoutManager layout)	Sets the layout manager
setVisible(boolean b)	Makes the frame visible/invisible
setBounds(int x, int y, int width, int height)	Sets position and size of frame
setDefaultCloseOperation(int operation)	Defines what happens when frame is closed

2. JFrame : Methods

Method	Purpose
add(Component c)	Adds a component to the frame
remove(Component c)	Removes a component from the frame
setResizable(boolean b)	Allows or restricts resizing
setLocation(int x, int y)	Sets position of frame on screen
getContentPane()	Returns content pane to add components
pack()	Adjusts size based on components

Example: Frame

```
import javax.swing.*;

public class Frame {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My First Frame");
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

2. JLabel (Text Display)

Syntax:

```
JLabel label = new JLabel("Text");  
label.setBounds(x, y, width, height);  
frame.add(label);
```

2. JLabel (Text Display)

Method	Purpose
setText(String text)	Sets the text of the label
getText()	Returns the label text
setBounds(int x, int y, int width, int height)	Sets position and size of label
setFont(Font f)	Changes the font style and size
setForeground(Color c)	Sets text color
setBackground(Color c)	Sets background color

2. JLabel (Text Display)

Method	Purpose
setOpaque(boolean b)	Makes background color visible
setHorizontalAlignment(int alignment)	Aligns text horizontally (LEFT, CENTER, RIGHT)
setVerticalAlignment(int alignment)	Aligns text vertically
setIcon(Icon icon)	Adds image/icon to label
setToolTipText(String text)	Shows tooltip text on hover
setOpaque(boolean b)	Makes background color visible

JAVA TECHNOLOGIES

```
import java.awt.*;
import javax.swing.*;
public class Frame {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JLabel Example");
        JLabel label = new JLabel("Hello Swing");
        label.setBounds(100, 100, 200, 30);
        label.setFont(new Font("Arial", Font.BOLD, 16));
        label.setForeground(Color.BLUE);
        frame.add(label);
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

4. JTextField : Single-line text input from the user.

Syntax:

```
JTextField textField = new JTextField();
```

Method	Description
setText(String)	Set text
getText()	Get entered text
setBounds(x,y,w,h)	Set size & position
setEditable(boolean)	Enable/disable editing

JAVA TECHNOLOGIES

```
import javax.swing.*;

public class TextFieldExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JTextField Example");
        JTextField tf = new JTextField();
        tf.setBounds(50, 50, 150, 30);

        frame.add(tf);
        frame.setSize(300, 200);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}
```

5. JTextArea : Multi-line text input.

Syntax:

```
JTextArea area = new JTextArea();
```

Method	Description
setText(String)	Set text
getText()	Get text
setLineWrap(true)	Wrap text automatically
setBounds(x,y,w,h)	Set position

JAVA TECHNOLOGIES

```
import javax.swing.*;

public class TextAreaExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JTextArea Example");
        JTextArea ta = new JTextArea();
        ta.setBounds(50, 50, 200, 100);

        frame.add(ta);
        frame.setSize(300, 250);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}
```

6. JCheckBox : Multiple selectable options.

Syntax:

```
JCheckBox cb = new JCheckBox("Option");
```

Method	Description
isSelected()	Check if selected
setSelected(boolean)	Set selection
setBounds(x,y,w,h)	Position & size
setText(String)	Change label
addActionListener()	Handle click event

6. JRadioButton : Only one option from a group.

Syntax:

```
JRadioButton rb = new JRadioButton("Option");
```

Method	Description
isSelected()	Check if selected
setSelected(boolean)	Set selection
setBounds(x,y,w,h)	Position & size
addActionListener()	Handle click

7. JComboBox : Drop-down list.

Syntax:

```
JComboBox<String> combo = new JComboBox<>();
```

Method	Description
addItem(String)	Add item
getSelectedItem()	Get selected item
setSelectedItem(String)	Set selected item
setBounds(x,y,w,h)	Set position
setEditable(true)	Allow typing

7. JList : One or multiple options.

Syntax:

```
JList<String> list = new JList<>(items);
```

Method	Description
getSelectedValue()	Get selected item
getSelectedValuesList()	Get multiple selected items
setSelectionMode()	Set single/multiple selection
setBounds(x,y,w,h)	Position & size

JAVA TECHNOLOGIES

Design a Student Registration Form using Java Swing that includes:

1. JTextField → Enter Name
2. JTextArea → Enter Address
3. JRadioButton → Select Gender
4. JCheckBox → Select Hobbies
5. JComboBox → Select Country
6. JList → Select Skills (multiple)
7. JButton → Submit form

When user clicks Submit, display all selected details.

JAVA TECHNOLOGIES

```
import javax.swing.*;  
import java.awt.event.*;  
import java.util.List;
```

```
public class StudentForm {  
    public static void main(String[] args) {
```

```
        JFrame frame = new JFrame("Student Registration Form");
```

```
        // Name
```

```
        JLabel l1 = new JLabel("Name:");  
        l1.setBounds(50, 30, 100, 30);  
        JTextField tf = new JTextField();  
        tf.setBounds(150, 30, 150, 30);
```

JAVA TECHNOLOGIES

// Address

```
JLabel l2 = new JLabel("Address:");  
l2.setBounds(50, 70, 100, 30);  
JTextArea ta = new JTextArea();  
ta.setBounds(150, 70, 150, 60);
```

// Gender

```
JLabel l3 = new JLabel("Gender:");  
l3.setBounds(50, 140, 100, 30);  
JRadioButton r1 = new JRadioButton("Male");  
JRadioButton r2 = new JRadioButton("Female");  
r1.setBounds(150, 140, 70, 30);  
r2.setBounds(220, 140, 80, 30);  
ButtonGroup bg = new ButtonGroup();  
bg.add(r1);  
bg.add(r2);
```

// Hobbies

```
JLabel l4 = new JLabel("Hobbies:");  
l4.setBounds(50, 180, 100, 30);  
JCheckBox cb1 = new JCheckBox("Reading");  
JCheckBox cb2 = new JCheckBox("Music");  
JCheckBox cb3 = new JCheckBox("Sports");  
cb1.setBounds(150, 180, 80, 30);  
cb2.setBounds(230, 180, 70, 30);  
cb3.setBounds(300, 180, 80, 30);
```

// Country

```
JLabel l5 = new JLabel("Country:");  
l5.setBounds(50, 220, 100, 30);  
String[] countries = {"India", "USA", "UK"};  
JComboBox<String> combo = new JComboBox<>(countries);  
combo.setBounds(150, 220, 120, 30);
```

// Skills

```
JLabel l6 = new JLabel("Skills:");  
l6.setBounds(50, 260, 100, 30);  
String[] skills = {"Java", "Python", "C++", "SQL"};  
JList<String> list = new JList<>(skills);  
list.setBounds(150, 260, 100, 80);
```

```
list.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION)  
;
```

// Submit Button

```
JButton btn = new JButton("Submit");  
btn.setBounds(150, 360, 100, 30);
```

// Action

```
btn.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        String name = tf.getText();  
        String address = ta.getText();  
        String gender = "";  
        if(r1.isSelected()) gender = "Male";  
        if(r2.isSelected()) gender = "Female";  
        String hobbies = "";  
        if(cb1.isSelected()) hobbies += "Reading ";  
        if(cb2.isSelected()) hobbies += "Music ";  
        if(cb3.isSelected()) hobbies += "Sports ";
```

JAVA TECHNOLOGIES

```
String country = (String) combo.getSelectedItem();
```

```
List<String> selectedSkills = list.getSelectedValuesList();
```

```
String result = "Name: " + name +  
    "\nAddress: " + address +  
    "\nGender: " + gender +  
    "\nHobbies: " + hobbies +  
    "\nCountry: " + country +  
    "\nSkills: " + selectedSkills;
```

```
JOptionPane.showMessageDialog(frame, result);
```

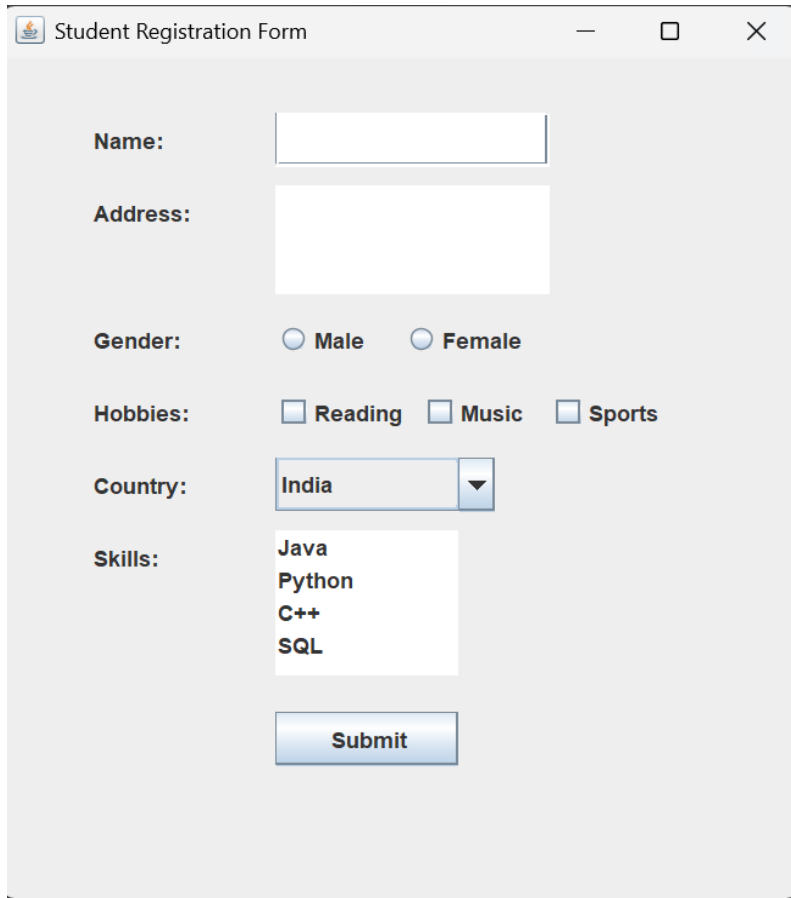
```
}  
});
```

// Add components

```
frame.add(l1); frame.add(tf);
frame.add(l2); frame.add(ta);
frame.add(l3); frame.add(r1); frame.add(r2);
frame.add(l4); frame.add(cb1); frame.add(cb2); frame.add(cb3);
frame.add(l5); frame.add(combo);
frame.add(l6); frame.add(list);
frame.add(btn);

frame.setSize(450, 500);
frame.setLayout(null);
frame.setVisible(true);
}
```

Output



Student Registration Form

Name:

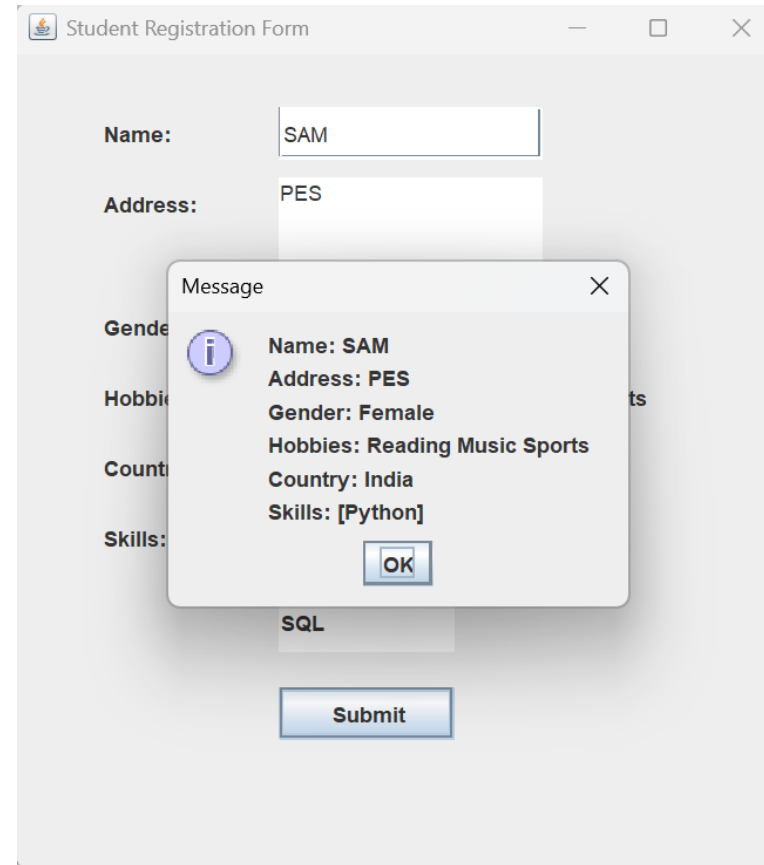
Address:

Gender: ☐ Male ☐ Female

Hobbies: ☐ Reading ☐ Music ☐ Sports

Country:

Skills:



Student Registration Form

Name:

Address:

Gender: ☐ Male ☐ Female

Hobbies: ☐ Reading ☐ Music ☐ Sports

Country:

Skills:

Message

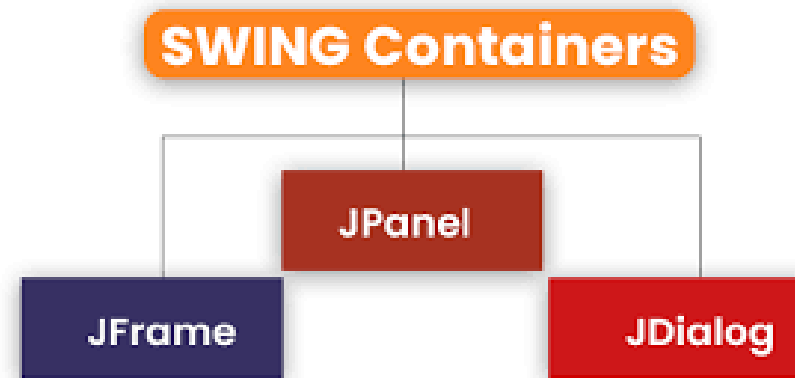
Name: SAM
Address: PES
Gender: Female
Hobbies: Reading Music Sports
Country: India
Skills: [Python]

Practice Program

Design a Java Swing application using Java Swing that:

- Uses multiple JPanels
- Performs simple operations (Addition & Subtraction)

Swing Containers



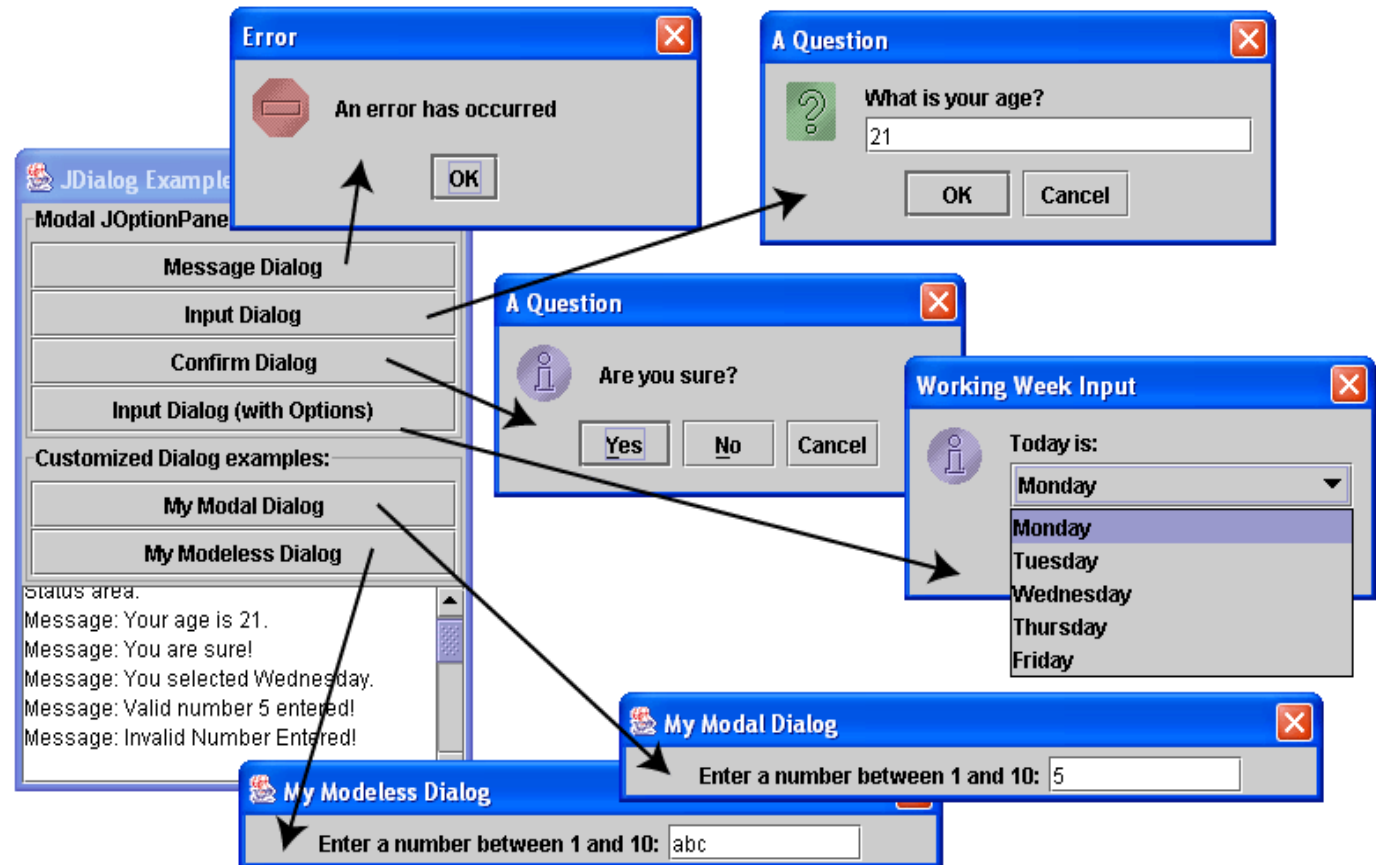
Container	Type	Purpose
JFrame	Top-level	Main window
JDialog	Top-level	Popup window
JPanel	Intermediate	Organize components

JDialog

- JDialog is a top-level container in Java Swing used to create temporary popup windows that interact with the user.
- It is always used along with a parent window (JFrame).
- It appears on top of the parent window
- The JDialog is very similar to a JFrame except that the JDialog can be set modally. Modal means that no other window can be used or activated while the corresponding JDialog is being displayed.
- Modal dialogs block input to other top-level windows and Modeless dialogs allow input to other windows.
- Unlike JFrame, JDialog does not hold minimize and maximize buttons at the top right corner of the window.

JDialog used to:

1. Show alerts (warnings, errors)
2. Take user input (login, form)
3. Ask confirmation (Yes/No)
4. Show settings/options



JDialog

Syntax:

```
JDialog dialog = new JDialog(frame, "Dialog", true);
```

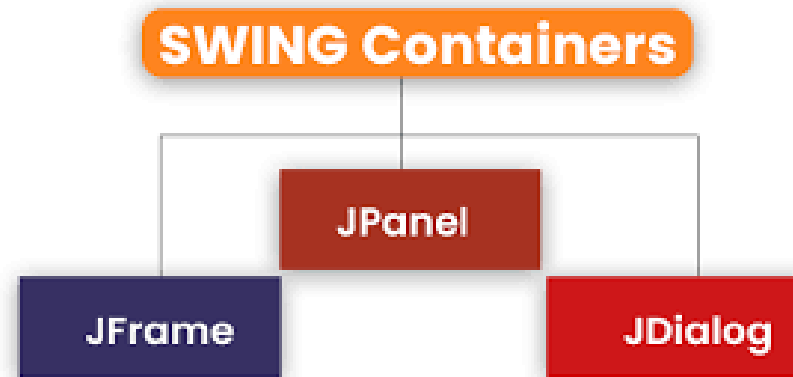
Lifecycle of JDialog

- Create dialog
- Add components (label, button, etc.)
- Set size/layout
- Make visible
- Close using dispose()

Jdialog : Methods

Method	Explanation
setModal(true)	Makes dialog blocking
setVisible(true)	Displays dialog
dispose()	Closes dialog and frees memory
setLocationRelativeTo()	Centers dialog
setDefaultCloseOperation()	Control closing behavior

Swing Containers



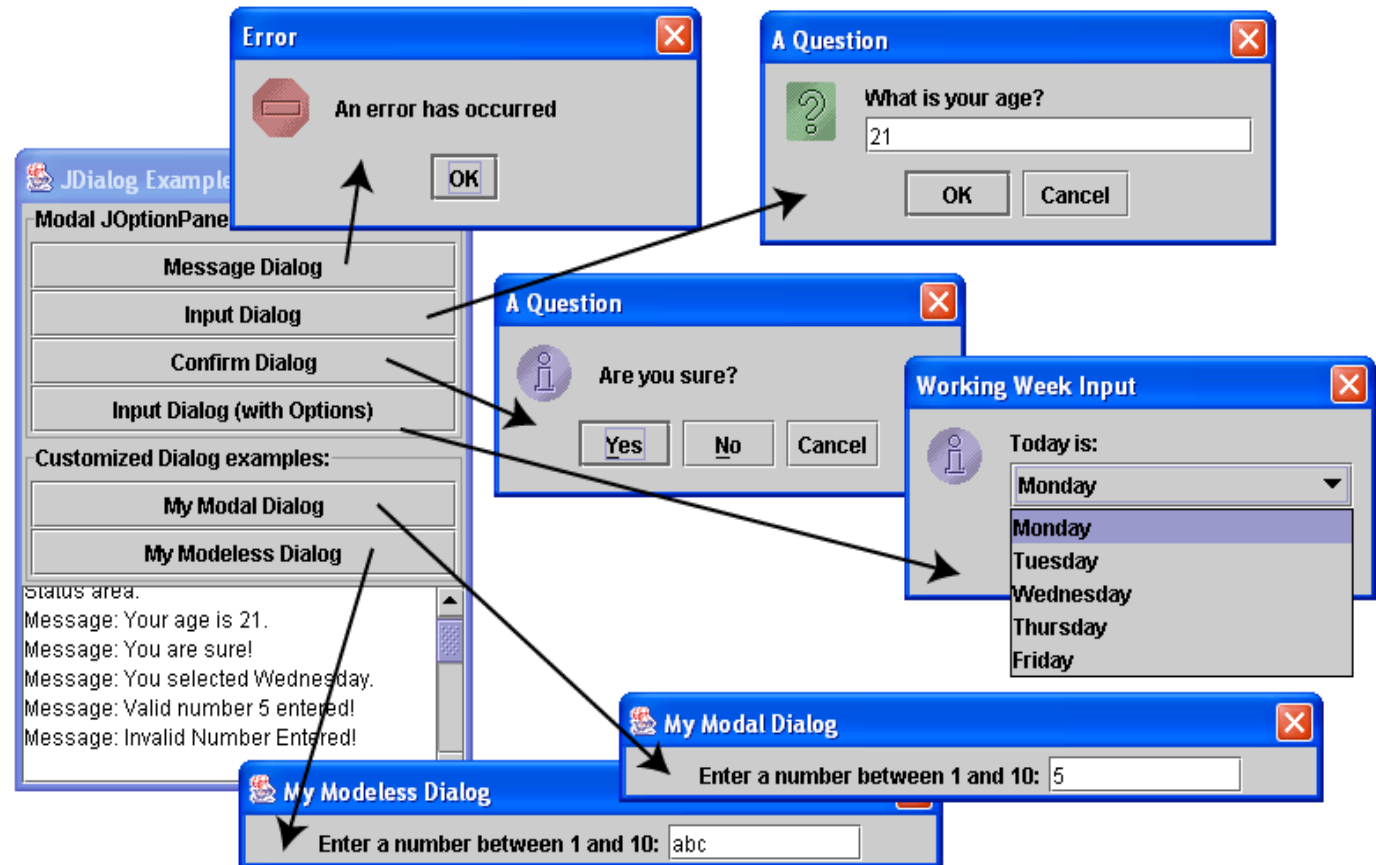
Container	Type	Purpose
JFrame	Top-level	Main window
JDialog	Top-level	Popup window
JPanel	Intermediate	Organize components

JDialog

- JDialog is a top-level container in Java Swing used to create temporary popup windows that interact with the user.
- It is always used along with a parent window (JFrame).
- It appears on top of the parent window
- The JDialog is very similar to a JFrame except that the JDialog can be set modally. Modal means that no other window can be used or activated while the corresponding JDialog is being displayed.
- Modal dialogs block input to other top-level windows and Modeless dialogs allow input to other windows.
- Unlike JFrame, JDialog does not hold minimize and maximize buttons at the top right corner of the window.

JDialog used to:

1. Show alerts (warnings, errors)
2. Take user input (login, form)
3. Ask confirmation (Yes/No)
4. Show settings/options



JDialog

Syntax:

```
JDialog dialog = new JDialog(frame, "Dialog", true);
```

Lifecycle of JDialog

- Create dialog
- Add components (label, button, etc.)
- Set size/layout
- Make visible
- Close using dispose()

Jdialog : Methods

Method	Explanation
setModal(true)	Makes dialog blocking
setVisible(true)	Displays dialog
dispose()	Closes dialog and frees memory
setLocationRelativeTo()	Centers dialog
setDefaultCloseOperation()	Control closing behavior

JAVA TECHNOLOGIES

Design a Java Swing application using Java Swing that demonstrates the use of JPanel.

The application should include:

A JFrame as the main window

Three JPanels arranged using BorderLayout:

- **Top Panel (NORTH) → Display a heading label**
- **Center Panel (CENTER) → Display a message label**
- **Bottom Panel (SOUTH) → Contain two buttons (OK and Cancel)**

Each panel should have a different background color

Properly display all panels inside the frame

JAVA TECHNOLOGIES



```
import javax.swing.*;
import java.awt.*;

public class PanelUnderstanding {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JPanel Demo");
        frame.setSize(400, 300);
        frame.setLayout(new BorderLayout());

        //Top Panel (Header)
        JPanel topPanel = new JPanel();
        topPanel.setBackground(Color.BLUE);
        JLabel title = new JLabel("TOP PANEL (Header)");
        title.setForeground(Color.WHITE);
        topPanel.add(title);
```

// Center Panel (Content)

```
JPanel centerPanel = new JPanel();  
centerPanel.setBackground(Color.GREEN);  
centerPanel.add(new JLabel("CENTER PANEL (Content Area)"));
```

// Bottom Panel (Buttons)

```
JPanel bottomPanel = new JPanel();  
bottomPanel.setBackground(Color.RED);  
bottomPanel.add(new JButton("OK"));  
bottomPanel.add(new JButton("Cancel"));
```

// Add panels to frame

```
frame.add(topPanel, BorderLayout.NORTH);  
frame.add(centerPanel, BorderLayout.CENTER);  
frame.add(bottomPanel, BorderLayout.SOUTH);
```

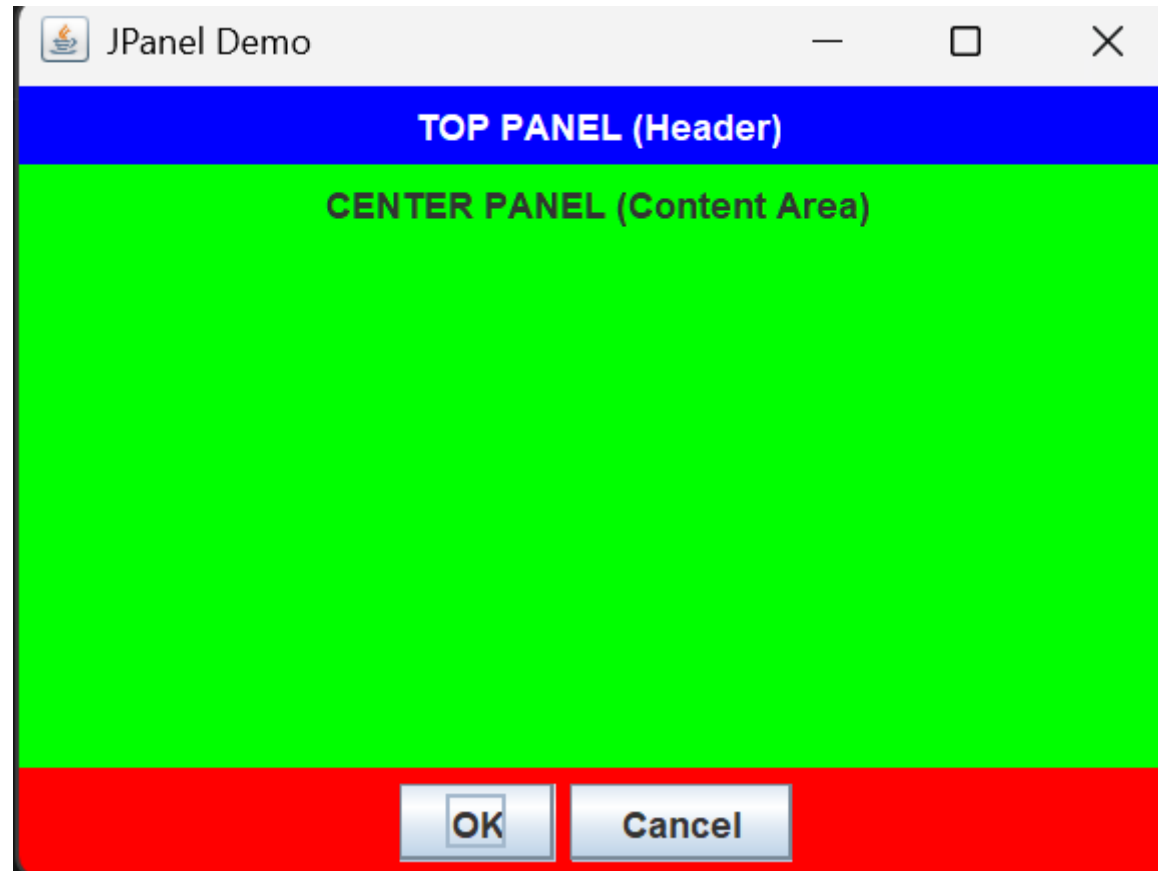
```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
frame.setVisible(true);
```

```
}
```

```
}
```

JAVA TECHNOLOGIES

Output:



Practice Program

Design a Java Swing application using Java Swing that:

- Uses multiple JPanels
- Performs simple operations (Addition & Subtraction)

Mouse Events

Mouse events are used to handle user interactions like clicking, moving, dragging, entering, or exiting a component using the mouse.

Types of Mouse Events

Event Type	Method	Description
Click Event	mouseClicked()	Triggered when mouse is clicked
Press Event	mousePressed()	When mouse button is pressed
Release Event	mouseReleased()	When mouse button is released
Enter Event	mouseEntered()	When mouse enters a component
Exit Event	mouseExited()	When mouse leaves a component
Move Event	mouseMoved()	When mouse moves
Drag Event	mouseDragged()	When mouse is dragged
Wheel Event	mouseWheelMoved()	When mouse wheel is scrolled

Mouse Listener Interfaces

Listener	Purpose
MouseListener	Handles click, press, release, enter, exit
MouseMotionListener	Handles move and drag
MouseWheelListener	Handles scroll wheel

Methods of Mouse Click

Method	Use
getX()	X-coordinate of mouse
getY()	Y-coordinate of mouse
getClickCount()	Number of clicks
getButton()	Which mouse button clicked

Example on Mouse Click Event

```
import javax.swing.*;  
import java.awt.event.*;  
  
public class MouseEventDemo {  
    public static void main(String[] args) {  
  
        JFrame f = new JFrame("Mouse Event Demo");  
        JLabel l = new JLabel("Click anywhere");  
        l.setBounds(100, 100, 200, 30);  
        f.add(l);  
    }  
}
```

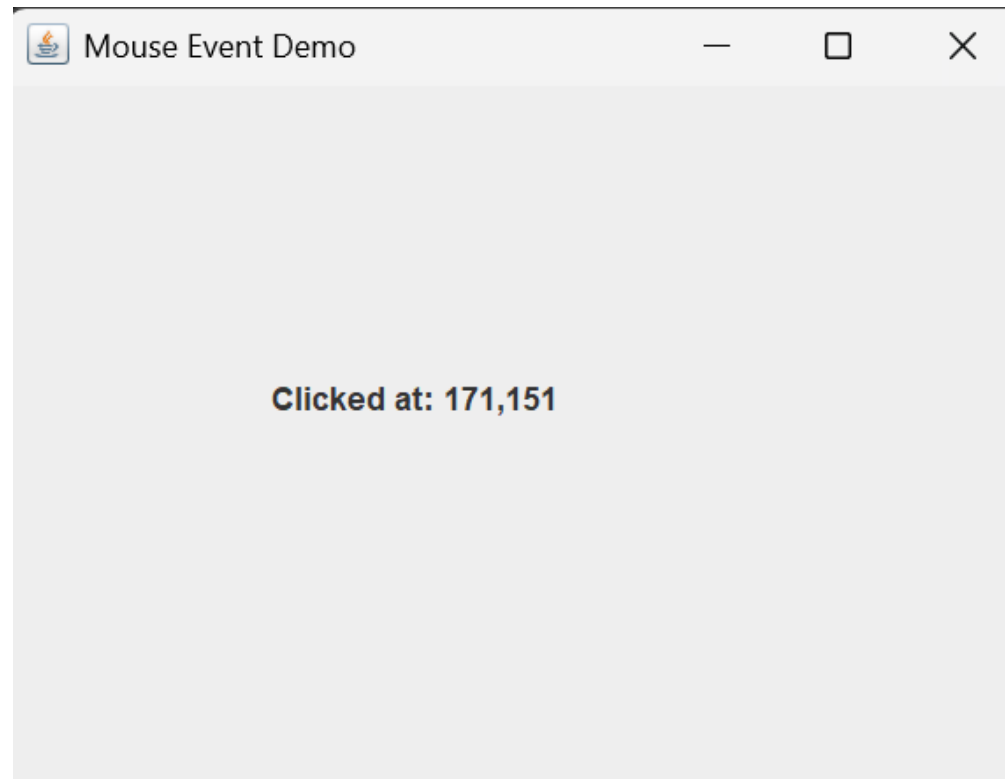
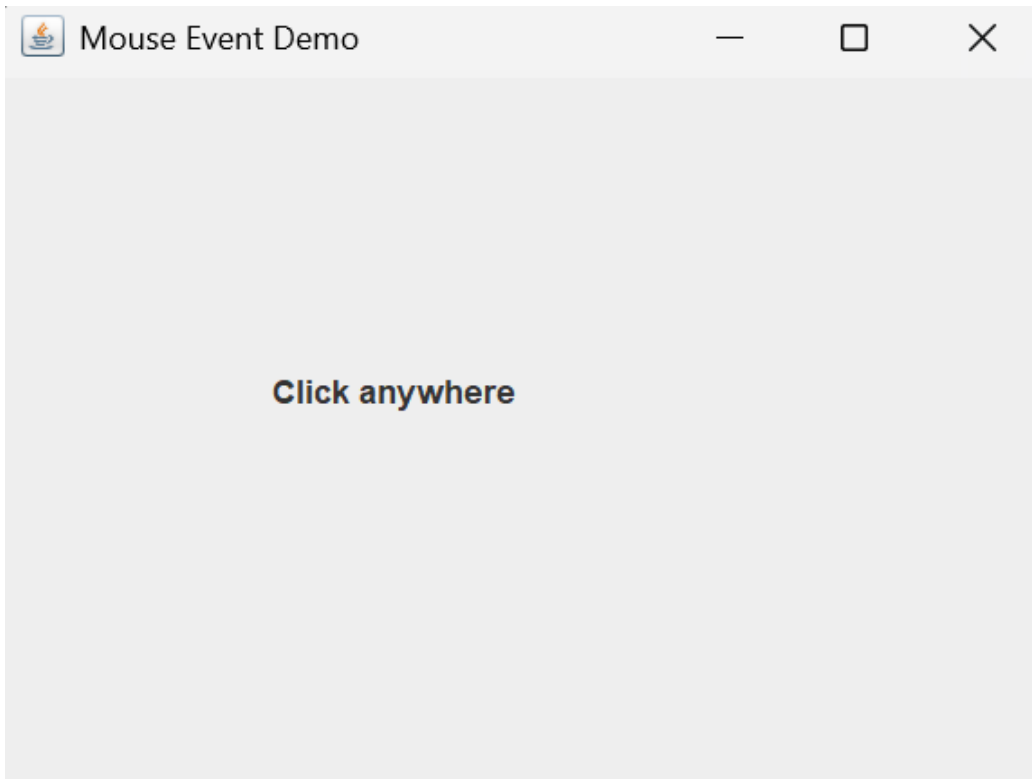
Example on Mouse Click Event

```
f.addMouseListener(new MouseAdapter() {  
    public void mouseClicked(MouseEvent e) {  
        l.setText("Clicked at: " + e.getX() + "," + e.getY());  
    }  
});
```

```
f.setSize(400, 300);  
f.setLayout(null);  
f.setVisible(true);  
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
}  
}
```

JAVA TECHNOLOGIES

Output: Mouse Click Event



Example on Mouse Click Event

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MouseEnterExitExample {
    public static void main(String[] args) {

        JFrame f = new JFrame("Mouse Enter Exit");

        f.addMouseListener(new MouseAdapter() {
            public void mouseEntered(MouseEvent e) {
                f.getContentPane().setBackground(Color.GREEN);
            }
        });
    }
}
```

Example on Mouse Click Event

```
public void mouseExited(MouseEvent e) {  
    f.getContentPane().setBackground(Color.RED);  
}  
});  
  
f.setSize(400, 300);  
f.setLayout(null);  
f.setVisible(true);  
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
}  
}
```

JAVA TECHNOLOGIES

Output:



Change background color when mouse enters/exits

Keyboard Events

Java Swing, keyboard events allow your application to respond to user input from the keyboard. These events are managed primarily through the **KeyListener** interface or, more modernly, through **Key Bindings**, attached to Swing components like JFrame, JTextField, JPanel, etc.

User presses key → Event generated → Listener handles it → Action performed

Work Flow of Keyboard Events

Keyboard Input



KeyEvent Object Created



Event Source (JFrame / JTextField)



KeyListener / KeyAdapter



keyPressed() / keyTyped() / keyReleased()



Program Action (UI update / logic)

Keyboard Events

Keyboard events can be handling

1. The KeyListener Interface

- Using KeyListener()
- Using KeyAdapter()

2. Key Bindings

Keyboard events

Method	Syntax	Key Points	Best Use
KeyListener	<code>addKeyListener(new KeyListener(){...})</code>	Interface with 3 methods: • <code>keyPressed()</code> • <code>keyReleased()</code> • <code>keyTyped()</code>	Learning fundamentals
KeyAdapter	<code>addKeyListener(new KeyAdapter(){...})</code>	Adapter class Override only required method	Simple Swing programs
Key Bindings	<code>InputMap + ActionMap</code>	Uses <code>KeyStroke</code> No direct listener methods	Professional applications

General Syntax for Keyboard Event (using KeyAdapter)

```
component.addKeyListener(new KeyAdapter() {  
    public void keyPressed(KeyEvent e) {  
        // code to execute when key is pressed  
    }  
});
```

Example:

```
addKeyListener(new KeyAdapter() {  
    public void keyPressed(KeyEvent e) {  
        System.out.println("Pressed: " + e.getKeyChar());  
    }  
});
```

1. The KeyListener Interface

The KeyListener is the traditional way to listen for keyboard activity. To use it, a component must be focusable and in focus.

Core Methods

The interface requires implementing three methods:

1. **keyTyped(KeyEvent e)**: Fired when a Unicode character is entered (e.g., typing 'a').
2. **keyPressed(KeyEvent e)**: Fired the moment a key is pressed down. This is the best place to catch action keys (arrows, modifiers).
3. **keyReleased(KeyEvent e)**: Fired when the user lifts their finger off a key.

2. Key Bindings

While `KeyListener` is simple, it has a major drawback: the component must have focus to work. If you have a game where a panel needs to move even if a button is clicked, `KeyListener` often fails.

Key Bindings solve this by mapping a `KeyStroke` to an `Action`, regardless of which specific sub-component has focus.

The Two-Map System

Key Bindings rely on two maps working together:

- **InputMap:** Maps a `KeyStroke` to an "Action Name" (a string).
- **ActionMap:** Maps that "Action Name" to an `Action` (the code to run).

Key Bindings- Map Types

Map Type	Description
WHEN_FOCUSED	Default; works only when the component has focus.
WHEN_IN_FOCUSED_WINDOW	Works if the component's window is active (most common for games).
WHEN_ANCESTOR_OF_FOCUSED_COMPONENT	Works if the component or any of its children has focus.

Key Differences: KeyListener vs. Key Bindings

Feature	KeyListener	Key Bindings
Focus Requirement	Component must have focus.	Can work globally within a window.
Implementation	Requires 3 methods (often empty).	Cleaner, decoupled Action logic.
Usability	Harder to manage in complex UIs.	Highly recommended for modern Swing.

Key Event Object

A **KeyEvent object** is created automatically when a key is:

- pressed
- released
- typed

KeyEvent Object is passed as a parameter to:

- keyPressed(KeyEvent e)
- keyReleased(KeyEvent e)
- keyTyped(KeyEvent e)

Here, e is the **KeyEvent object**

Information Inside KeyEvent Object

Method	Description
e.getKeyChar()	Returns the character (e.g., 'A', 'b')
e.getKeyCode()	Returns integer key code
e.getKeyText(int code)	Returns key name (e.g., "Enter")
e.getModifiersEx()	Returns modifier keys (Shift, Ctrl, etc.)
e.isShiftDown()	Checks if Shift is pressed
e.isControlDown()	Checks if Ctrl is pressed
e.isAltDown()	Checks if Alt is pressed

Pre-Defined Constants

Key	Constant
Enter	KeyEvent.VK_ENTER
Escape	KeyEvent.VK_ESCAPE
Up Arrow	KeyEvent.VK_UP
Down Arrow	KeyEvent.VK_DOWN
Left Arrow	KeyEvent.VK_LEFT
Right Arrow	KeyEvent.VK_RIGHT

Example: Java Program on Keyboard Event- keyPressed

```
import javax.swing.*;
import java.awt.event.*;

public class KeyDemo {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Keyboard Event");

        frame.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                System.out.println("Pressed: " + e.getKeyChar());
            }
        });
    }
}
```

JAVA TECHNOLOGIES

Cont..

```
frame.setSize(400, 300);
    frame.setLayout(null);
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    frame.setFocusable(true); // important
}
```

Example: Program on Keyboard Events

```
import javax.swing.*;
import java.awt.event.*;
public class AllKeyEventsDemo extends JFrame {
    JLabel label;
    AllKeyEventsDemo() {
        label = new JLabel("Press any key...");
        label.setBounds(50, 100, 300, 30);
        add(label);
        addKeyListener(new KeyAdapter() {
        }
```

Cont..

// 1. Key Pressed

```
public void keyPressed(KeyEvent e) {  
    label.setText("Key Pressed: " + KeyEvent.getKeyText(e.getKeyCode())); }
```

// 2. Key Released

```
public void keyReleased(KeyEvent e) {  
    label.setText("Key Released: " + KeyEvent.getKeyText(e.getKeyCode())); }
```

// 3. Key Typed

```
public void keyTyped(KeyEvent e) {  
    label.setText("Key Typed: " + e.getKeyChar()); }  
});
```

Cont..

```
setSize(400, 300);
setLayout(null);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setFocusable(true);
requestFocus();
}
public static void main(String[] args) {
    new AllKeyEventsDemo();
}
}
```

JAVA TECHNOLOGIES

Example: Write Program to Detects special keys like Enter, Arrow keys, Escape

```
import javax.swing.*;
import java.awt.event.*;

public class KeyConstantsExample extends JFrame {
    JLabel label;

    KeyConstantsExample() {
        label = new JLabel("Press special keys...");
        label.setBounds(50, 100, 300, 30);
        add(label);
        addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                int key = e.getKeyCode();
                if (key == KeyEvent.VK_ENTER) {
                    label.setText("Enter Key Pressed");
                }
            }
        });
    }
}
```

Cont..

```
else if (key == KeyEvent.VK_UP) {  
    label.setText("Up Arrow Pressed");  
}  
else if (key == KeyEvent.VK_DOWN) {  
    label.setText("Down Arrow Pressed");  
}  
else if (key == KeyEvent.VK_LEFT) {  
    label.setText("Left Arrow Pressed");  
}  
else if (key == KeyEvent.VK_RIGHT) {  
    label.setText("Right Arrow Pressed");  
}  
else if (key == KeyEvent.VK_ESCAPE) {  
    label.setText("Escape Key Pressed");  
}  
else {  
    label.setText("Other Key Pressed");  
}  
});
```

Cont..

```
setSize(400, 300);
setLayout(null);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setFocusable(true);
requestFocus();
}

public static void main(String[] args) {
    new KeyConstantsExample();
}
}
```

JAVA TECHNOLOGIES

Example : Program prints a message when Enter key is pressed

```
import java.awt.event.*;
import javax.swing.*;

public class KeyBindingDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Key Binding Example");
        JPanel panel = new JPanel();

        // Define key stroke
        KeyStroke key = KeyStroke.getKeyStroke("ENTER");

        // Map key to action name
        panel.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW)
            .put(key, "enterAction");
    }
}
```

Cont..

// Define action

```
panel.getActionMap().put("enterAction", new AbstractAction() {  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Enter Key Pressed!");  
    }  
});
```

```
frame.add(panel);  
frame.setSize(400, 300);  
frame.setVisible(true);  
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
}  
}
```



PES
UNIVERSITY

CELEBRATING 50 YEARS

THANK YOU

Samyukta D Kumta

Department of Computer Applications

samyuktad@pes.edu